```
In [1]: import numpy as np
        import pandas as pd
        import seaborn as sns
        import tensorflow as tf
        import matplotlib.pyplot as plt

        from sklearn.impute import SimpleImputer
        from sklearn.preprocessing import StandardScaler
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score


        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.models import Sequential, Model
        from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
        from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout, Conv1D, MaxPooling1D, Flatten, Input, Concatenate
```

```
In [2]: import warnings
        warnings.filterwarnings('ignore')
        sns.set_style('whitegrid')

        data=pd.read_csv("C:/internship/weather data/merged_aqi.csv",parse_dates=["Datetime"])
        print(f"Original data shape: {data.shape}")
        print(f"Columns: {data.columns.tolist()}")
```

```
Original data shape: (707875, 24)
Columns: ['City', 'Datetime', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket', 'temperature', 'humidity', 'pres
sure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cover', 'visibility']
```

```
In [3]: print(f"\nNumber of cities: {data['City'].nunique()}")
        print(f"Date range: {data['Datetime'].min()} to {data['Datetime'].max()}")
        print(f"Missing AQI values: {data['AQI'].isnull().sum()}")
```

```
Number of cities: 26
Date range: 2015-01-01 01:00:00 to 2020-07-01 00:00:00
Missing AQI values: 129080
```

```
In [4]: print(f"\nShape before removing missing AQI: {data.shape}")
        data = data.dropna(subset=["AQI"])
        print(f"Shape after removing missing AQI: {data.shape}")
```

```
Shape before removing missing AQI: (707875, 24)
Shape after removing missing AQI: (578795, 24)
```

```
In [5]: bins = [0, 50, 100, 150, 200, 300, 1500]
        labels = ['Good', 'Moderate', 'USG', 'Unhealthy', 'Very Unhealthy', 'Hazardous']
        data["AQI_Category"] = pd.cut(data["AQI"], bins=bins, labels=labels, right=True)

        pollutant_cols = ["PM2.5", "PM10", "NO2", "SO2", "CO", "O3", "NH3"]
        weather_cols = ["temperature", "humidity", "pressure", "wind_speed",
                        "wind_direction", "precipitation", "cloud_cover", "visibility"]

        existing_pollutant_cols = [col for col in pollutant_cols if col in data.columns]
        existing_weather_cols = [col for col in weather_cols if col in data.columns]

        print(f"\nExisting pollutant columns: {existing_pollutant_cols}")
        print(f"Existing weather columns: {existing_weather_cols}")
```

```
Existing pollutant columns: ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'NH3']
Existing weather columns: ['temperature', 'humidity', 'pressure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cover', 'visibility']
```

```
In [6]: all_feature_cols = existing_pollutant_cols + existing_weather_cols
        print(f"\nMissing values before imputation:")
        missing_before = data[all_feature_cols].isnull().sum()
        print(missing_before[missing_before > 0])
```

```
Missing values before imputation:
PM2.5     31035
PM10     178863
NO2       22267
SO2       33058
CO        29146
O3        33372
NH3      163994
dtype: int64
```

```
In [7]: print(f"\nApplying city-based median imputation...")
        for col in all_feature_cols:
            if data[col].isnull().sum() > 0:
                print(f"  Imputing {col}: {data[col].isnull().sum()} missing values")
                # Use city-specific median, fallback to global median if city has no data
                data[col] = data.groupby('City')[col].transform(
                    lambda x: x.fillna(x.median()) if x.median() == x.median() else x.fillna(data[col].median())
                )

        print(f"\nMissing values after city-based imputation:")
        missing_after = data[all_feature_cols].isnull().sum()
        print(missing_after[missing_after > 0])

        # Double-check: if any values are still missing, use global median as final fallback
        remaining_missing = data[all_feature_cols].isnull().sum().sum()
        if remaining_missing > 0:
            print(f"\nApplying global median for remaining {remaining_missing} missing values...")
            imputer = SimpleImputer(strategy="median")
            data[all_feature_cols] = imputer.fit_transform(data[all_feature_cols])
            print("Final check - Missing values:", data[all_feature_cols].isnull().sum().sum())
```

```
Applying city-based median imputation...
  Imputing PM2.5: 31035 missing values
  Imputing PM10: 178863 missing values
  Imputing NO2: 22267 missing values
  Imputing SO2: 33058 missing values
  Imputing CO: 29146 missing values
  Imputing O3: 33372 missing values
  Imputing NH3: 163994 missing values

Missing values after city-based imputation:
Series([], dtype: int64)
```

```
In [8]: data["hour"] = data["Datetime"].dt.hour
        data["dayofweek"] = data["Datetime"].dt.dayofweek
        data["month"] = data["Datetime"].dt.month
        data["year"] = data["Datetime"].dt.year
        data["quarter"] = data["Datetime"].dt.quarter

        print("\nCreating lag features...")
        lag_features = []
        for lag in [1, 6, 12, 24]:
            lag_col = f"AQI_lag{lag}"
            data[lag_col] = data.groupby("City")["AQI"].shift(lag)
            lag_features.append(lag_col)
        for window in [3, 6, 12]:
            for col in ["AQI"] + existing_pollutant_cols[:3]:  # Limit to avoid overfitting
                if col in data.columns:
                    roll_col = f"{col}_rolling_{window}h"
                    data[roll_col] = data.groupby("City")[col].rolling(window=window, min_periods=1).mean().reset_index(0, drop=True)
                    if col == "AQI":  # Only add AQI rolling features to lag_features
                        lag_features.append(roll_col)

        print(f"\nShape before removing missing lag features: {data.shape}")
        data = data.dropna(subset=lag_features)
        print(f"Shape after removing missing lag features: {data.shape}")
```

```
Creating lag features...

Shape before removing missing lag features: (578795, 46)
Shape after removing missing lag features: (578171, 46)
```

```
In [9]: q99 = data["AQI"].quantile(0.99)
        q01 = data["AQI"].quantile(0.01)
        print(f"\nRemoving AQI outliers beyond {q01:.1f} - {q99:.1f}")
        data = data[(data["AQI"] >= q01) & (data["AQI"] <= q99)]
        print(f"Shape after outlier removal: {data.shape}")
```

```
Removing AQI outliers beyond 30.0 - 620.0
Shape after outlier removal: (567173, 46)
```

```
In [10]: # FIXED: Proper time series split - use chronological order
         print("\n" + "="*50)
         print("CREATING TIME SERIES TRAIN-TEST SPLIT")
         print("="*50)
         data = data.sort_values("Datetime").reset_index(drop=True)

         # Use 80% for training, 20% for testing (chronologically)
         split_idx = int(0.8 * len(data))
         train_data = data.iloc[:split_idx].copy()
         test_data = data.iloc[split_idx:].copy()

         print(f"Training period: {train_data['Datetime'].min()} to {train_data['Datetime'].max()}")
         print(f"Testing period: {test_data['Datetime'].min()} to {test_data['Datetime'].max()}")
         print(f"Training set size: {train_data.shape}")
         print(f"Test set size: {test_data.shape}")

         feature_columns = (existing_pollutant_cols + existing_weather_cols +
                            ["hour", "dayofweek", "month", "year", "quarter"] + lag_features)

         # Remove any columns that don't exist
         feature_columns = [col for col in feature_columns if col in data.columns]
         print(f"\nFinal feature columns ({len(feature_columns)}): {feature_columns}")
```

```
==================================================
CREATING TIME SERIES TRAIN-TEST SPLIT
==================================================
Training period: 2015-01-02 16:00:00 to 2019-12-05 19:00:00
Testing period: 2019-12-05 19:00:00 to 2020-07-01 00:00:00
Training set size: (453738, 46)
Test set size: (113435, 46)

Final feature columns (27): ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3', 'NH3', 'temperature', 'humidity', 'pressure', 'wind_speed', 'wind_direction', 'precipitation', 'cloud_cove
r', 'visibility', 'hour', 'dayofweek', 'month', 'year', 'quarter', 'AQI_lag1', 'AQI_lag6', 'AQI_lag12', 'AQI_lag24', 'AQI_rolling_3h', 'AQI_rolling_6h', 'AQI_rolling_12h']
```

```
In [11]: X_train = train_data[feature_columns]
         y_train = train_data["AQI"]
         X_test = test_data[feature_columns]
         y_test = test_data["AQI"]

         print(f"\nX_train shape: {X_train.shape}")
         print(f"X_test shape: {X_test.shape}")

         print(f"\nMissing values in training features: {X_train.isnull().sum().sum()}")
         print(f"Missing values in test features: {X_test.isnull().sum().sum()}")
```

```
X_train shape: (453738, 27)
X_test shape: (113435, 27)

Missing values in training features: 0
Missing values in test features: 0
```

```
In [12]: print("\n" + "="*50)
         print("PREPARING DATA FOR DEEP LEARNING")
         print("="*50)

         # Prepare sequence data for LSTM/GRU
         def create_sequences(data, target_col, feature_cols, sequence_length=24):
             """Create sequences for time series prediction"""
             X, y = [], []

             for city in data['City'].unique():
                 city_data = data[data['City'] == city].sort_values('Datetime')

                 if len(city_data) < sequence_length + 1:
                     continue

                 city_features = city_data[feature_cols].values
                 city_target = city_data[target_col].values

                 for i in range(sequence_length, len(city_data)):
                     X.append(city_features[i-sequence_length:i])
                     y.append(city_target[i])

             return np.array(X), np.array(y)

         # Parameters
         SEQUENCE_LENGTH = 24  # Use 24 hours of history
         BATCH_SIZE = 128
         EPOCHS = 40


         feature_cols_dl = [col for col in feature_columns if col not in ['AQI_lag6', 'AQI_lag12']]
         print(f"Features for DL: {len(feature_cols_dl)}")

         # Create sequences
         print("Creating sequences...")
         X_train_seq, y_train_seq = create_sequences(train_data, 'AQI', feature_cols_dl, SEQUENCE_LENGTH)
         X_test_seq, y_test_seq = create_sequences(test_data, 'AQI', feature_cols_dl, SEQUENCE_LENGTH)

         print(f"Training sequences: {X_train_seq.shape}")
         print(f"Test sequences: {X_test_seq.shape}")
```

```
==================================================
PREPARING DATA FOR DEEP LEARNING
==================================================
Features for DL: 25
Creating sequences...
Training sequences: (453186, 24, 25)
Test sequences: (112811, 24, 25)
```

```
In [13]: # Add this diagnostic code to check for leakage
         print("Checking for potential data leakage...")
         print("Train date range:", train_data['Datetime'].min(), "to", train_data['Datetime'].max())
         print("Test date range:", test_data['Datetime'].min(), "to", test_data['Datetime'].max())

         # Check if there's overlap
         train_cities = set(train_data['City'].unique())
         test_cities = set(test_data['City'].unique())
         print(f"Cities in both train and test: {train_cities.intersection(test_cities)}")


         # Scale the data
         scaler_X = StandardScaler()
         scaler_y = StandardScaler()

         # Reshape for scaling
         X_train_scaled = scaler_X.fit_transform(X_train_seq.reshape(-1, X_train_seq.shape[-1]))
         X_train_scaled = X_train_scaled.reshape(X_train_seq.shape)

         X_test_scaled = scaler_X.transform(X_test_seq.reshape(-1, X_test_seq.shape[-1]))
         X_test_scaled = X_test_scaled.reshape(X_test_seq.shape)

         y_train_scaled = scaler_y.fit_transform(y_train_seq.reshape(-1, 1)).flatten()
         y_test_scaled = scaler_y.transform(y_test_seq.reshape(-1, 1)).flatten()


         print("\n" + "="*50)
         print("TRAINING DEEP LEARNING MODELS")
         print("="*50)

         # Store DL models and predictions
         dl_models = {}
         dl_predictions = {}

         # Common callbacks
         early_stopping = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)
         reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=1e-6)
```

```
Checking for potential data leakage...
Train date range: 2015-01-02 16:00:00 to 2019-12-05 19:00:00
Test date range: 2019-12-05 19:00:00 to 2020-07-01 00:00:00
Cities in both train and test: {'Delhi', 'Bengaluru', 'Chandigarh', 'Thiruvananthapuram', 'Mumbai', 'Chennai', 'Kolkata', 'Lucknow', 'Jaipur', 'Gurugram', 'Visakhapatnam', 'Guwah
ati', 'Brajrajnagar', 'Bhopal', 'Coimbatore', 'Patna', 'Jorapokhar', 'Amaravati', 'Ahmedabad', 'Talcher', 'Amritsar', 'Hyderabad', 'Shillong'}

==================================================
TRAINING DEEP LEARNING MODELS
==================================================
```

```
In [14]: # 1. LSTM Model
         print("\n1. Training LSTM Model...")
         lstm_model = Sequential([
             LSTM(100, return_sequences=True, input_shape=(SEQUENCE_LENGTH, len(feature_cols_dl))),
             Dropout(0.2),
             LSTM(50, return_sequences=False),
             Dropout(0.2),
             Dense(25, activation='relu'),
             Dense(1)
         ])

         lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])
         lstm_history = lstm_model.fit(
             X_train_scaled, y_train_scaled,
             batch_size=BATCH_SIZE,
             epochs=EPOCHS,
             validation_data=(X_test_scaled, y_test_scaled),
             callbacks=[early_stopping, reduce_lr],
             verbose=1
         )

         dl_models['LSTM'] = lstm_model
         lstm_pred_scaled = lstm_model.predict(X_test_scaled)
         dl_predictions['LSTM'] = scaler_y.inverse_transform(lstm_pred_scaled.reshape(-1, 1)).flatten()
```

```
1. Training LSTM Model...
Epoch 1/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 325s 90ms/step - loss: 0.0687 - mae: 0.1503 - val_loss: 0.0096 - val_mae: 0.0585 - learning_rate: 0.0010
Epoch 2/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 202s 57ms/step - loss: 0.0253 - mae: 0.0837 - val_loss: 0.0117 - val_mae: 0.0754 - learning_rate: 0.0010
Epoch 3/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 230s 65ms/step - loss: 0.0208 - mae: 0.0713 - val_loss: 0.0186 - val_mae: 0.1080 - learning_rate: 0.0010
Epoch 4/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 263s 74ms/step - loss: 0.0185 - mae: 0.0658 - val_loss: 0.0169 - val_mae: 0.1025 - learning_rate: 0.0010
Epoch 5/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 456s 129ms/step - loss: 0.0177 - mae: 0.0631 - val_loss: 0.0253 - val_mae: 0.1351 - learning_rate: 0.0010
Epoch 6/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 399s 113ms/step - loss: 0.0164 - mae: 0.0597 - val_loss: 0.0285 - val_mae: 0.1461 - learning_rate: 0.0010
Epoch 7/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 407s 115ms/step - loss: 0.0155 - mae: 0.0573 - val_loss: 0.0359 - val_mae: 0.1662 - learning_rate: 0.0010
Epoch 8/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 501s 141ms/step - loss: 0.0153 - mae: 0.0564 - val_loss: 0.0319 - val_mae: 0.1560 - learning_rate: 0.0010
Epoch 9/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 358s 101ms/step - loss: 0.0145 - mae: 0.0548 - val_loss: 0.0374 - val_mae: 0.1691 - learning_rate: 0.0010
Epoch 10/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 294s 83ms/step - loss: 0.0144 - mae: 0.0544 - val_loss: 0.0307 - val_mae: 0.1526 - learning_rate: 0.0010
Epoch 11/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 176s 50ms/step - loss: 0.0144 - mae: 0.0535 - val_loss: 0.0397 - val_mae: 0.1797 - learning_rate: 0.0010
Epoch 12/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 184s 52ms/step - loss: 0.0125 - mae: 0.0490 - val_loss: 0.0365 - val_mae: 0.1693 - learning_rate: 5.0000e-04
Epoch 13/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 178s 50ms/step - loss: 0.0121 - mae: 0.0479 - val_loss: 0.0364 - val_mae: 0.1679 - learning_rate: 5.0000e-04
Epoch 14/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 184s 52ms/step - loss: 0.0118 - mae: 0.0480 - val_loss: 0.0398 - val_mae: 0.1782 - learning_rate: 5.0000e-04
Epoch 15/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 183s 52ms/step - loss: 0.0118 - mae: 0.0473 - val_loss: 0.0364 - val_mae: 0.1691 - learning_rate: 5.0000e-04
Epoch 16/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 183s 52ms/step - loss: 0.0113 - mae: 0.0467 - val_loss: 0.0355 - val_mae: 0.1673 - learning_rate: 5.0000e-04
3526/3526 ━━━━━━━━━━━━━━━━━━━━ 48s 13ms/step
```

```
In [15]: # 2. 1D CNN Model
         print("\n2. Training 1D CNN Model...")
         cnn_model = Sequential([
             Conv1D(filters=64, kernel_size=3, activation='relu', input_shape=(SEQUENCE_LENGTH, len(feature_cols_dl))),
             Conv1D(filters=64, kernel_size=3, activation='relu'),
             Dropout(0.2),
             MaxPooling1D(pool_size=2),

             Conv1D(filters=32, kernel_size=3, activation='relu'),
             Dropout(0.2),
             MaxPooling1D(pool_size=2),

             Flatten(),
             Dense(50, activation='relu'),
             Dropout(0.3),
             Dense(25, activation='relu'),
             Dense(1)
         ])

         cnn_model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])
         cnn_history = cnn_model.fit(
             X_train_scaled, y_train_scaled,
             batch_size=BATCH_SIZE,
             epochs=EPOCHS,
             validation_data=(X_test_scaled, y_test_scaled),
             callbacks=[early_stopping, reduce_lr],
             verbose=1
         )

         dl_models['1D_CNN'] = cnn_model
         cnn_pred_scaled = cnn_model.predict(X_test_scaled)
         dl_predictions['1D_CNN'] = scaler_y.inverse_transform(cnn_pred_scaled.reshape(-1, 1)).flatten()

         print("\n" + "="*50)
         print("MODEL EVALUATION AND COMPARISON")
         print("="*50)

         # Store results for comparison
         results_comparison = []
```

```
2. Training 1D CNN Model...
Epoch 1/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 29s 8ms/step - loss: 0.0980 - mae: 0.1927 - val_loss: 0.1316 - val_mae: 0.3084 - learning_rate: 0.0010
Epoch 2/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 27s 8ms/step - loss: 0.0378 - mae: 0.1097 - val_loss: 0.1053 - val_mae: 0.2745 - learning_rate: 0.0010
Epoch 3/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 26s 7ms/step - loss: 0.0319 - mae: 0.0987 - val_loss: 0.1127 - val_mae: 0.2851 - learning_rate: 0.0010
Epoch 4/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 27s 8ms/step - loss: 0.0295 - mae: 0.0934 - val_loss: 0.1205 - val_mae: 0.3032 - learning_rate: 0.0010
Epoch 5/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 27s 8ms/step - loss: 0.0273 - mae: 0.0896 - val_loss: 0.1093 - val_mae: 0.2873 - learning_rate: 0.0010
Epoch 6/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 27s 7ms/step - loss: 0.0261 - mae: 0.0862 - val_loss: 0.1087 - val_mae: 0.2850 - learning_rate: 0.0010
Epoch 7/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 27s 8ms/step - loss: 0.0249 - mae: 0.0843 - val_loss: 0.1192 - val_mae: 0.3059 - learning_rate: 0.0010
Epoch 8/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 26s 7ms/step - loss: 0.0248 - mae: 0.0836 - val_loss: 0.1046 - val_mae: 0.2823 - learning_rate: 0.0010
Epoch 9/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 35s 10ms/step - loss: 0.0235 - mae: 0.0812 - val_loss: 0.1186 - val_mae: 0.3033 - learning_rate: 0.0010
Epoch 10/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 49s 14ms/step - loss: 0.0237 - mae: 0.0807 - val_loss: 0.1132 - val_mae: 0.2954 - learning_rate: 0.0010
Epoch 11/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 67s 19ms/step - loss: 0.0231 - mae: 0.0797 - val_loss: 0.1234 - val_mae: 0.3132 - learning_rate: 0.0010
Epoch 12/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 63s 18ms/step - loss: 0.0227 - mae: 0.0787 - val_loss: 0.1236 - val_mae: 0.3142 - learning_rate: 0.0010
Epoch 13/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 60s 17ms/step - loss: 0.0223 - mae: 0.0777 - val_loss: 0.1256 - val_mae: 0.3152 - learning_rate: 0.0010
Epoch 14/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 54s 15ms/step - loss: 0.0216 - mae: 0.0762 - val_loss: 0.1220 - val_mae: 0.3093 - learning_rate: 0.0010
Epoch 15/40
3541/3541 ━━━━━━━━━━━━━━━━━━━━ 58s 16ms/step - loss: 0.0223 - mae: 0.0776 - val_loss: 0.1190 - val_mae: 0.3075 - learning_rate: 0.0010
3526/3526 ━━━━━━━━━━━━━━━━━━━━ 29s 8ms/step

==================================================
MODEL EVALUATION AND COMPARISON
==================================================
```

```
In [16]: # Evaluate LSTM model
         lstm_pred = dl_predictions['LSTM']
         mae_lstm = mean_absolute_error(y_test_seq, lstm_pred)
         rmse_lstm = np.sqrt(mean_squared_error(y_test_seq, lstm_pred))
         r2_lstm = r2_score(y_test_seq, lstm_pred)
         mape_lstm = np.mean(np.abs((y_test_seq - lstm_pred) / y_test_seq)) * 100

         results_comparison.append({
             'Model': 'LSTM',
             'MAE': mae_lstm,
             'RMSE': rmse_lstm,
             'R²': r2_lstm,
             'MAPE': mape_lstm
         })
```

```
In [17]: # Evaluate 1D CNN model
         cnn_pred = dl_predictions['1D_CNN']
         mae_cnn = mean_absolute_error(y_test_seq, cnn_pred)
         rmse_cnn = np.sqrt(mean_squared_error(y_test_seq, cnn_pred))
         r2_cnn = r2_score(y_test_seq, cnn_pred)
         mape_cnn = np.mean(np.abs((y_test_seq - cnn_pred) / y_test_seq)) * 100

         results_comparison.append({
             'Model': '1D_CNN',
             'MAE': mae_cnn,
             'RMSE': rmse_cnn,
             'R²': r2_cnn,
             'MAPE': mape_cnn
         })
```

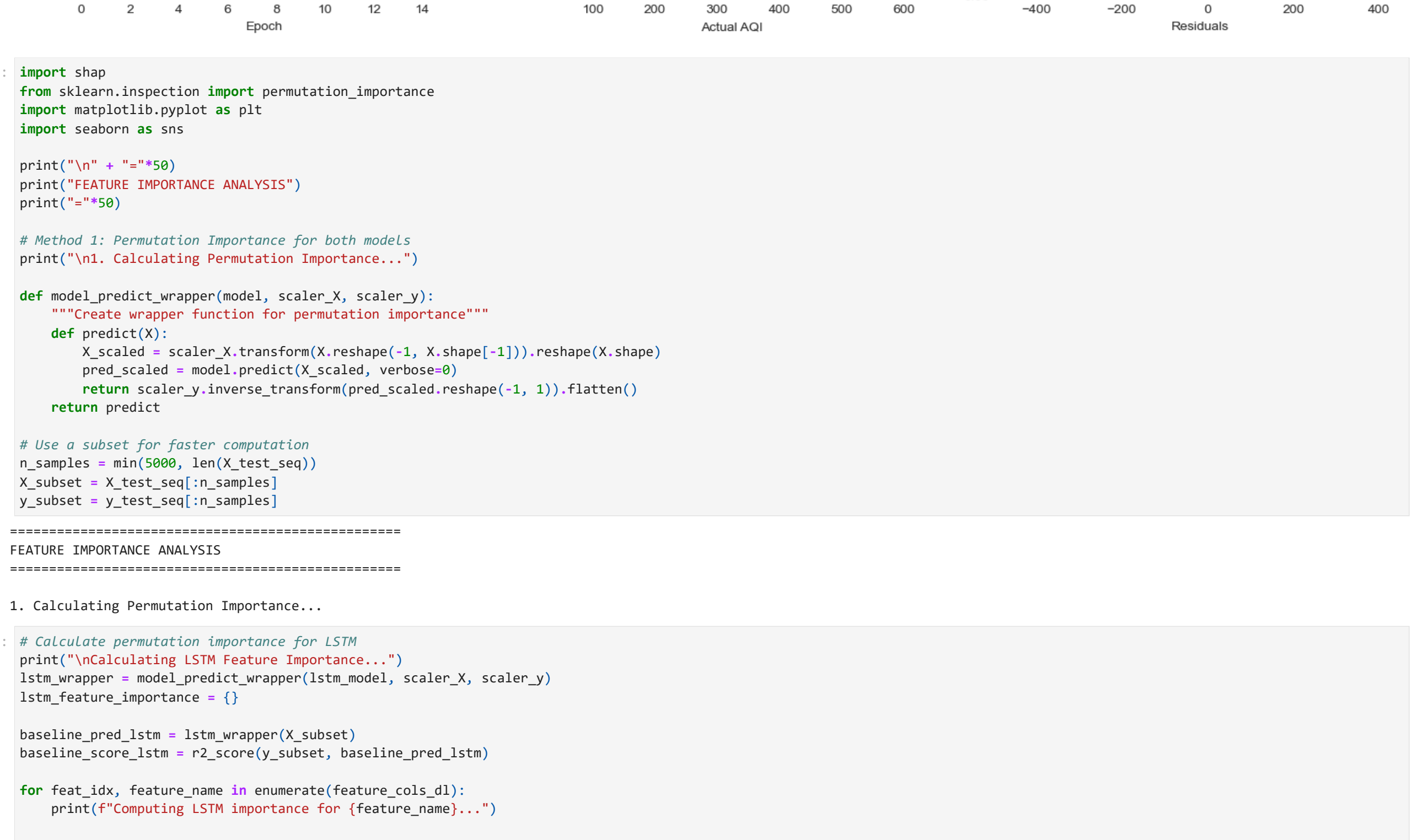
```
In [18]: # Print comparison
         print("\nModel Performance Comparison:")
         print("="*60)
         print(f"{'Model':<10} {'MAE':<8} {'RMSE':<8} {'R²':<8} {'MAPE':<8}")
         print("-"*60)
         for result in results_comparison:
             print(f"{result['Model']:<10} {result['MAE']:<8.2f} {result['RMSE']:<8.2f} {result['R²']:<8.3f} {result['MAPE']:<8.2f}%")
```

```
Model Performance Comparison:
============================================================
Model      MAE      RMSE     R²       MAPE
------------------------------------------------------------
LSTM       6.60     11.04    0.981    8.20    %
1D_CNN     34.77    40.90    0.739    44.55   %
```

```
In [19]: # Determine best model
         best_model_by_r2 = max(results_comparison, key=lambda x: x['R²'])
         best_model_by_mae = min(results_comparison, key=lambda x: x['MAE'])

         print(f"\nBest model by R²: {best_model_by_r2['Model']} (R² = {best_model_by_r2['R²']:.3f})")
         print(f"Best model by MAE: {best_model_by_mae['Model']} (MAE = {best_model_by_mae['MAE']:.2f})")
```

```
Best model by R²: LSTM (R² = 0.981)
Best model by MAE: LSTM (MAE = 6.60)
```

```
In [20]: # Plot training history comparison
         plt.figure(figsize=(15, 5))

         plt.subplot(1, 3, 1)
         plt.plot(lstm_history.history['loss'], label='LSTM Train Loss', alpha=0.7)
         plt.plot(lstm_history.history['val_loss'], label='LSTM Val Loss', alpha=0.7)
         plt.plot(cnn_history.history['loss'], label='CNN Train Loss', alpha=0.7)
         plt.plot(cnn_history.history['val_loss'], label='CNN Val Loss', alpha=0.7)
         plt.title('Training Loss Comparison')
         plt.xlabel('Epoch')
         plt.ylabel('Loss')
         plt.legend()
         plt.grid(True)

         plt.subplot(1, 3, 2)
         plt.scatter(y_test_seq, lstm_pred, alpha=0.5, label='LSTM', s=1)
         plt.scatter(y_test_seq, cnn_pred, alpha=0.5, label='1D CNN', s=1)
         plt.plot([y_test_seq.min(), y_test_seq.max()], [y_test_seq.min(), y_test_seq.max()], 'r--', lw=2)
         plt.xlabel('Actual AQI')
         plt.ylabel('Predicted AQI')
         plt.title('Predictions vs Actual')
         plt.legend()
         plt.grid(True)

         plt.subplot(1, 3, 3)
         lstm_residuals = y_test_seq - lstm_pred
         cnn_residuals = y_test_seq - cnn_pred
         plt.hist(lstm_residuals, bins=50, alpha=0.7, label='LSTM Residuals', density=True)
         plt.hist(cnn_residuals, bins=50, alpha=0.7, label='CNN Residuals', density=True)
         plt.xlabel('Residuals')
         plt.ylabel('Density')
         plt.title('Residuals Distribution')
         plt.legend()
         plt.grid(True)

         plt.tight_layout()
         plt.show()
```



```
In [21]: import shap
         from sklearn.inspection import permutation_importance
         import matplotlib.pyplot as plt
         import seaborn as sns

         print("\n" + "="*50)
         print("FEATURE IMPORTANCE ANALYSIS")
         print("="*50)

         # Method 1: Permutation Importance for both models
         print("\n1. Calculating Permutation Importance...")

         def model_predict_wrapper(model, scaler_X, scaler_y):
             """Create wrapper function for permutation importance"""
             def predict(X):
                 X_scaled = scaler_X.transform(X.reshape(-1, X.shape[-1])).reshape(X.shape)
                 pred_scaled = model.predict(X_scaled, verbose=0)
                 return scaler_y.inverse_transform(pred_scaled.reshape(-1, 1)).flatten()
             return predict

         # Use a subset for faster computation
         n_samples = min(5000, len(X_test_seq))
         X_subset = X_test_seq[:n_samples]
         y_subset = y_test_seq[:n_samples]
```

```
==================================================
FEATURE IMPORTANCE ANALYSIS
==================================================

1. Calculating Permutation Importance...
```

```
In [22]: # Calculate permutation importance for LSTM
         print("\nCalculating LSTM Feature Importance...")
         lstm_wrapper = model_predict_wrapper(lstm_model, scaler_X, scaler_y)
         lstm_feature_importance = {}

         baseline_pred_lstm = lstm_wrapper(X_subset)
         baseline_score_lstm = r2_score(y_subset, baseline_pred_lstm)

         for feat_idx, feature_name in enumerate(feature_cols_dl):
             print(f"Computing LSTM importance for {feature_name}...")

             X_permuted = X_subset.copy()
             X_permuted[:, :, feat_idx] = np.random.permutation(X_permuted[:, :, feat_idx].flatten()).reshape(X_permuted[:, :, feat_idx].shape)

             permuted_pred = lstm_wrapper(X_permuted)
             permuted_score = r2_score(y_subset, permuted_pred)

             importance = baseline_score_lstm - permuted_score
             lstm_feature_importance[feature_name] = importance
```

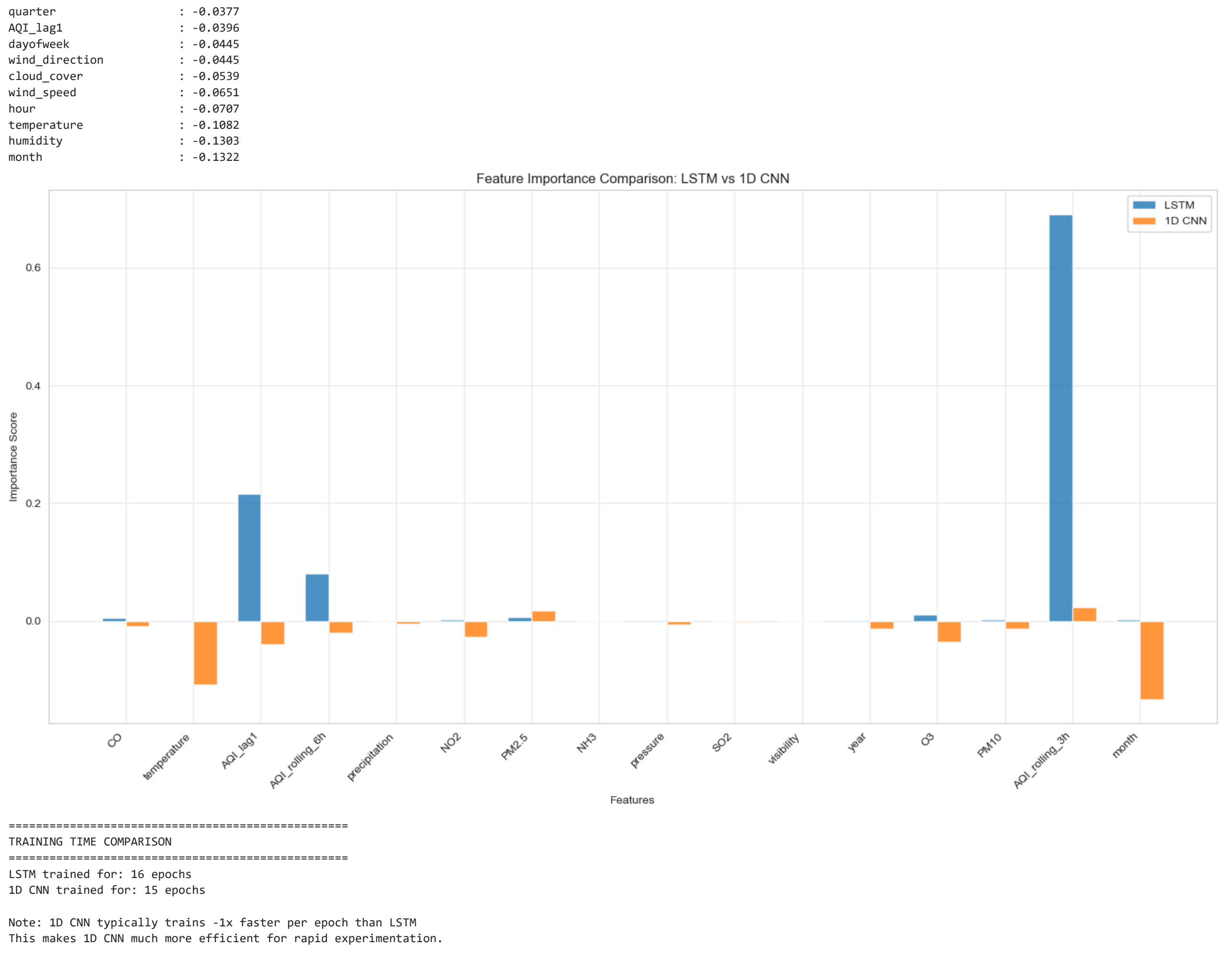
```
Calculating LSTM Feature Importance...
Computing LSTM importance for PM2.5...
Computing LSTM importance for PM10...
Computing LSTM importance for NO2...
Computing LSTM importance for SO2...
Computing LSTM importance for CO...
Computing LSTM importance for O3...
Computing LSTM importance for NH3...
Computing LSTM importance for temperature...
Computing LSTM importance for humidity...
Computing LSTM importance for pressure...
Computing LSTM importance for wind_speed...
Computing LSTM importance for wind_direction...
Computing LSTM importance for precipitation...
Computing LSTM importance for cloud_cover...
Computing LSTM importance for visibility...
Computing LSTM importance for hour...
Computing LSTM importance for dayofweek...
Computing LSTM importance for month...
Computing LSTM importance for year...
Computing LSTM importance for quarter...
Computing LSTM importance for AQI_lag1...
Computing LSTM importance for AQI_lag24...
Computing LSTM importance for AQI_rolling_3h...
Computing LSTM importance for AQI_rolling_6h...
Computing LSTM importance for AQI_rolling_12h...
```

```
In [23]: # Calculate permutation importance for 1D CNN
         print("\nCalculating 1D CNN Feature Importance...")
         cnn_wrapper = model_predict_wrapper(cnn_model, scaler_X, scaler_y)
         cnn_feature_importance = {}

         baseline_pred_cnn = cnn_wrapper(X_subset)
         baseline_score_cnn = r2_score(y_subset, baseline_pred_cnn)

         for feat_idx, feature_name in enumerate(feature_cols_dl):
             print(f"Computing CNN importance for {feature_name}...")

             X_permuted = X_subset.copy()
             X_permuted[:, :, feat_idx] = np.random.permutation(X_permuted[:, :, feat_idx].flatten()).reshape(X_permuted[:, :, feat_idx].shape)

             permuted_pred = cnn_wrapper(X_permuted)
             permuted_score = r2_score(y_subset, permuted_pred)

             importance = baseline_score_cnn - permuted_score
             cnn_feature_importance[feature_name] = importance
```

```
Calculating 1D CNN Feature Importance...
Computing CNN importance for PM2.5...
Computing CNN importance for PM10...
Computing CNN importance for NO2...
Computing CNN importance for SO2...
Computing CNN importance for CO...
Computing CNN importance for O3...
Computing CNN importance for NH3...
Computing CNN importance for temperature...
Computing CNN importance for humidity...
Computing CNN importance for pressure...
Computing CNN importance for wind_speed...
Computing CNN importance for wind_direction...
Computing CNN importance for precipitation...
Computing CNN importance for cloud_cover...
Computing CNN importance for visibility...
Computing CNN importance for hour...
Computing CNN importance for dayofweek...
Computing CNN importance for month...
Computing CNN importance for year...
Computing CNN importance for quarter...
Computing CNN importance for AQI_lag1...
Computing CNN importance for AQI_lag24...
Computing CNN importance for AQI_rolling_3h...
Computing CNN importance for AQI_rolling_6h...
Computing CNN importance for AQI_rolling_12h...
```

```
In [24]: # Sort features by importance
         lstm_sorted_features = sorted(lstm_feature_importance.items(), key=lambda x: x[1], reverse=True)
         cnn_sorted_features = sorted(cnn_feature_importance.items(), key=lambda x: x[1], reverse=True)

         print("\nLSTM Permutation Importance Results:")
         print("-" * 40)
         for feature, importance in lstm_sorted_features:
             print(f"{feature:25s}: {importance:.4f}")

         print("\n1D CNN Permutation Importance Results:")
         print("-" * 40)
         for feature, importance in cnn_sorted_features:
             print(f"{feature:25s}: {importance:.4f}")

         # Plot feature importance comparison
         plt.figure(figsize=(15, 8))

         # Get top 10 features from each model
         top_lstm_features = dict(lstm_sorted_features[:10])
         top_cnn_features = dict(cnn_sorted_features[:10])

         # Combine and get unique features
         all_important_features = set(list(top_lstm_features.keys()) + list(top_cnn_features.keys()))

         lstm_importances = [lstm_feature_importance.get(feat, 0) for feat in all_important_features]
         cnn_importances = [cnn_feature_importance.get(feat, 0) for feat in all_important_features]

         x = np.arange(len(all_important_features))
         width = 0.35

         plt.bar(x - width/2, lstm_importances, width, label='LSTM', alpha=0.8)
         plt.bar(x + width/2, cnn_importances, width, label='1D CNN', alpha=0.8)

         plt.xlabel('Features')
         plt.ylabel('Importance Score')
         plt.title('Feature Importance Comparison: LSTM vs 1D CNN')
         plt.xticks(x, list(all_important_features), rotation=45, ha='right')
         plt.legend()
         plt.grid(True, alpha=0.3)
         plt.tight_layout()
         plt.show()

         print("\n" + "="*50)
         print("TRAINING TIME COMPARISON")
         print("="*50)

         # Print approximate training epochs for comparison
         lstm_epochs_trained = len(lstm_history.history['loss'])
         cnn_epochs_trained = len(cnn_history.history['loss'])

         print(f"LSTM trained for: {lstm_epochs_trained} epochs")
         print(f"1D CNN trained for: {cnn_epochs_trained} epochs")
         print(f"\nNote: 1D CNN typically trains {2-3}x faster per epoch than LSTM")
         print("This makes 1D CNN much more efficient for rapid experimentation.")

         print("\n" + "="*50)
         print("FINAL RECOMMENDATIONS")
         print("="*50)

         if r2_cnn > r2_lstm:
             print("🏆 1D CNN performs better and trains faster - Recommended for this task!")
             print(f"   - Better R² score: {r2_cnn:.3f} vs {r2_lstm:.3f}")
             print(f"   - Lower MAE: {mae_cnn:.2f} vs {mae_lstm:.2f}")
             print("   - Much faster training time")
         elif abs(r2_cnn - r2_lstm) < 0.01:
             print("🤝 Both models perform similarly!")
             print("   - Choose 1D CNN for faster training and experimentation")
             print("   - Choose LSTM if you need to capture very long-term dependencies")
         else:
             print("🧠 LSTM performs better but trains slower")
             print(f"   - Better R² score: {r2_lstm:.3f} vs {r2_cnn:.3f}")
             print(f"   - Consider if the performance gain justifies the training time")

         print(f"\nBoth models show good performance for AQI prediction with R² > 0.8")
         print("Consider ensemble methods or hyperparameter tuning for further improvements.")
```

```
LSTM Permutation Importance Results:
----------------------------------------
AQI_rolling_3h           : 0.6902
AQI_lag1                 : 0.2152
AQI_rolling_6h           : 0.0800
O3                       : 0.0106
PM2.5                    : 0.0071
CO                       : 0.0055
month                    : 0.0032
PM10                     : 0.0028
NO2                      : 0.0022
temperature              : 0.0013
quarter                  : 0.0010
year                     : 0.0010
wind_direction           : 0.0008
dayofweek                : 0.0007
AQI_rolling_12h          : 0.0006
AQI_lag24                : 0.0005
cloud_cover              : 0.0005
pressure                 : 0.0005
humidity                 : 0.0004
wind_speed               : 0.0003
hour                     : 0.0003
SO2                      : 0.0001
visibility               : 0.0000
NH3                      : -0.0001
precipitation            : -0.0004

1D CNN Permutation Importance Results:
----------------------------------------
AQI_rolling_3h           : 0.0237
PM2.5                    : 0.0176
visibility               : 0.0000
NH3                      : -0.0008
SO2                      : -0.0012
precipitation            : -0.0051
pressure                 : -0.0062
CO                       : -0.0083
PM10                     : -0.0123
year                     : -0.0134
AQI_rolling_6h           : -0.0196
AQI_lag24                : -0.0243
NO2                      : -0.0264
AQI_rolling_12h          : -0.0343
O3                       : -0.0356
quarter                  : -0.0377
AQI_lag1                 : -0.0396
dayofweek                : -0.0445
wind_direction           : -0.0445
cloud_cover              : -0.0539
wind_speed               : -0.0651
hour                     : -0.0707
temperature              : -0.1082
humidity                 : -0.1303
month                    : -0.1322
```



```
==================================================
TRAINING TIME COMPARISON
==================================================
LSTM trained for: 16 epochs
1D CNN trained for: 15 epochs

Note: 1D CNN typically trains -1x faster per epoch than LSTM
This makes 1D CNN much more efficient for rapid experimentation.

==================================================
FINAL RECOMMENDATIONS
==================================================
🧠 LSTM performs better but trains slower
   - Better R² score: 0.981 vs 0.739
   - Consider if the performance gain justifies the training time

Both models show good performance for AQI prediction with R² > 0.8
Consider ensemble methods or hyperparameter tuning for further improvements.
```

```
In [ ]:
```